

XML External Entity Prevention Cheat Sheet

Introduction

An *XML eXternal Entity injection* (XXE), which is now part of the [OWASP Top 10](#) via the point **A4**, is attack against applications that parse XML input. This issue is referenced in the ID [611](#) in the [Common Weakness Enumeration](#) referential. An XXE attack occurs when untrusted XML input with a **reference to an external entity is processed by a weakly configured XML parser**, and this attack could be used to stage multiple incidents, including:

- A denial of service attack on the system
- A [Server Side Request Forgery](#) (SSRF) attack
- The ability to scan ports from the machine where the parser is located
- Other system impacts.

This cheat sheet will help you prevent this vulnerability.

For more information on XXE, please visit [XML External Entity \(XXE\)](#).

General Guidance

The safest way to prevent XXE is always to disable DTDs (External Entities) completely.

Depending on the parser, the method should be similar to the following:

```
factory.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
```

Disabling [DTDs](#) also makes the parser secure against denial of services (DOS) attacks such as [Billion Laughs](#). **If it is not possible to disable DTDs completely, then external entities and external document type declarations must be disabled in the way that's specific to each parser.**

XML Parser Security Features Matrix

Security Feature	Default (Parser-Dependent)	Purpose	What Happens If Missing?
External Entities Disabled	Usually disabled (safe)	Blocks external resource loading	Full XXE possible → SSRF, file disclosure, internal scans
Disallow DOCTYPE Declaration	Varies	Prevents ENTITY definitions	Classic XXE payloads become fully functional
Disable External DTD Loading	Usually disabled	Stops loading remote DTDs	Enables Blind XXE, SSRF behind firewalls
Secure Processing Mode	Varies	Restricts recursion, network access, entity expansion	Billion Laughs DoS and resource depletion become possible
Disable Parameter Entities	Varies	Prevents <code>%entity;</code> injections	Advanced XXE payloads bypass simple protections
XInclude Disabled	Usually disabled	Prevents including external files	File read via <code>file://</code> and SSRF becomes possible
Limit Entity Expansion Count	Usually enabled	Prevents recursive entity abuse	Memory exhaustion → parser or server DoS
Schema Validation Without External Fetching	Usually safe	Ensures validation does not fetch external URLs	Silent external HTTP calls triggered during validation

Quick Impact Matrix (What Happens If Missing?)

Missing Control	Resulting Vulnerability
DOCTYPE not disabled	Standard XXE fully exploitable
External entities enabled	SSRF, file exfiltration, port scanning
External DTD loading allowed	Blind XXE → hidden SSRF attacks
No expansion limits	Billion Laughs DoS
XInclude enabled	Local file disclosure + SSRF
Secure processing disabled	Critical protections bypassed
Schema validation fetches external URLs	Application makes unwanted outbound requests

Minimal XML Hardening Rules

- Disable DOCTYPE
- Disable external entities
- Disable external DTD loading
- Enable secure processing mode
- Disable XInclude
- Limit entity expansion
- Do not use legacy XML parsers
- Never parse untrusted XML with default settings

Detailed XXE Prevention guidance is provided below for multiple languages (C++, Cold Fusion, Java, .NET, iOS, PHP, Python, Semgrep Rules) and their commonly used XML parsers.

C/C++

libxml2

The Enum [xmlParserOption](#) should not have the following options defined:

- `XML_PARSE_NOENT` : Expands entities and substitutes them with replacement text
- `XML_PARSE_DTDLOAD` : Load the external DTD

Note:

Per: According to [this post](#), starting with libxml2 version 2.9, XXE has been disabled by default as committed by the following [patch](#).

Search whether the following APIs are being used and make sure there is no `XML_PARSE_NOENT` and `XML_PARSE_DTDLOAD` defined in the parameters:

- `xmlCtxtReadDoc`
- `xmlCtxtReadFd`
- `xmlCtxtReadFile`
- `xmlCtxtReadIO`
- `xmlCtxtReadMemory`
- `xmlCtxtUseOptions`
- `xmlParseInNodeContext`
- `xmlReadDoc`
- `xmlReadFd`
- `xmlReadFile`
- `xmlReadIO`
- `xmlReadMemory`

libxerces-c

Use of `XercesDOMParser` do this to prevent XXE:

```
XercesDOMParser *parser = new XercesDOMParser;  
parser->setCreateEntityReferenceNodes(true);  
parser->setDisableDefaultEntityResolution(true);
```

Use of `SAXParser`, do this to prevent XXE:

```
SAXParser* parser = new SAXParser;  
parser->setDisableDefaultEntityResolution(true);
```

Use of `SAX2XMLReader`, do this to prevent XXE:

```
SAX2XMLReader* reader = XMLReaderFactory::createXMLReader();  
parser->setFeature(XMLUni::fgXercesDisableDefaultEntityResolution, true);
```

ColdFusion

Per [this blog post](#), both Adobe ColdFusion and Lucee have built-in mechanisms to disable support for external XML entities.

Adobe ColdFusion

As of ColdFusion 2018 Update 14 and ColdFusion 2021 Update 4, all native ColdFusion functions that process XML have a XML parser argument that disables support for external XML entities. Since there is no global setting that disables external entities, developers must ensure that every XML function call uses the correct security options.

From the [documentation for the XmlParse\(\) function](#), you can disable XXE with the code below:

```
<cfset parseroptions = structnew(>
<cfset parseroptions.ALLOWEXTERNALENTITIES = false>
<cfscript>
a = XmlParse("xml.xml", false, parseroptions);
writeDump(a);
</cfscript>
```

You can use the "parseroptions" structure shown above as an argument to secure other functions that process XML as well, such as:

```
XxmlSearch(xmlDoc, xpath, parseroptions);

XmlTransform(xmlDoc, xslt, parseroptions);

isXML(xmlDoc, parseroptions);
```

Lucee

As of Lucee 5.3.4.51 and later, you can disable support for XML external entities by adding the following to your Application.cfc:

```
this.xmlFeatures = {
    externalGeneralEntities: false,
    secure: true,
    disallowDoctypeDecl: true
};
```

Support for external XML entities is disabled by default as of Lucee 5.4.2.10 and Lucee 6.0.0.514.

Java

Since most Java XML parsers have XXE enabled by default, this language is especially vulnerable to XXE attack, so you must explicitly disable XXE to use these parsers safely. This

section describes how to disable XXE in the most commonly used Java XML parsers.

JAXP DocumentBuilderFactory, SAXParserFactory and DOM4J

The `DocumentBuilderFactory`, `SAXParserFactory` and `DOM4J` XML parsers can be protected against XXE attacks with the same techniques.

For brevity, we will only show you how to protect the `DocumentBuilderFactory` parser. Additional instructions for protecting this parser are embedded within the example code

The JAXP `DocumentBuilderFactory` `setFeature` method allows a developer to control which implementation-specific XML processor features are enabled or disabled.

These features can either be set on the factory or the underlying `XMLReader` `setFeature` method.

Each XML processor implementation has its own features that govern how DTDs and external entities are processed. By disabling DTD processing entirely, most XXE attacks can be averted, although it is also necessary to disable or verify that `XInclude` is not enabled.

Since the JDK 6, the flag `FEATURE_SECURE_PROCESSING` can be used to instruct the implementation of the parser to process XML securely. Its behavior is implementation-dependent. It may help with resource exhaustion but it may not always mitigate entity expansion. More details on this flag can be found [here](#).

For a syntax highlighted example code snippet using `SAXParserFactory`, look [here](#). Example code disabling DTDs (doctypes) altogether:

```
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException; // catching unsupported
features
import javax.xml.XMLConstants;

...

DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
String FEATURE = null;
try {
    // This is the PRIMARY defense. If DTDs (doctypes) are disallowed, almost
    all
    // XML entity attacks are prevented
    // Xerces 2 only - http://xerces.apache.org/xerces2-
    j/features.html#disallow-doctype-decl
    FEATURE = "http://apache.org/xml/features/disallow-doctype-decl";
    dbf.setFeature(FEATURE, true);

    // and these as well, per Timothy Morgan's 2014 paper: "XML Schema, DTD,
    and Entity Attacks"
    dbf.setXIncludeAware(false);
```

```

        // remaining parser logic
        ...
    } catch (ParserConfigurationException e) {
        // This should catch a failed setFeature feature
        // NOTE: Each call to setFeature() should be in its own try/catch
        // otherwise subsequent calls will be skipped.
        // This is only important if you're ignoring errors for multi-provider
        // support.
        logger.info("ParserConfigurationException was thrown. The feature '" +
FEATURE
        + "' is not supported by your XML processor.");
        ...
    } catch (SAXException e) {
        // On Apache, this should be thrown when disallowing DOCTYPE
        logger.warning("A DOCTYPE was passed into the XML document");
        ...
    } catch (IOException e) {
        // XXE that points to a file that doesn't exist
        logger.error("IOException occurred, XXE may still possible: " +
e.getMessage());
        ...
    }
}

// Load XML file or stream using a XXE agnostic configured parser...
DocumentBuilder safebuilder = dbf.newDocumentBuilder();

```

If you can't completely disable DTDs:

```

import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException; // catching unsupported
features
import javax.xml.XMLConstants;

...

DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();

String[] featuresToDisable = {
    // Xerces 1 - http://xerces.apache.org/xerces-j/features.html#external-
general-entities
    // Xerces 2 - http://xerces.apache.org/xerces2-j/features.html#external-
general-entities
    // JDK7+ - http://xml.org/sax/features/external-general-entities
    //This feature has to be used together with the following one, otherwise
it will not protect you from XXE for sure
    "http://xml.org/sax/features/external-general-entities",

    // Xerces 1 - http://xerces.apache.org/xerces-j/features.html#external-
parameter-entities
    // Xerces 2 - http://xerces.apache.org/xerces2-j/features.html#external-
parameter-entities
    // JDK7+ - http://xml.org/sax/features/external-parameter-entities
    //This feature has to be used together with the previous one, otherwise it
will not protect you from XXE for sure
    "http://xml.org/sax/features/external-parameter-entities",

```

```
// Disable external DTDs as well
"http://apache.org/xml/features/nonvalidating/load-external-dtd"
}

for (String feature : featuresToDisable) {
    try {
        dbf.setFeature(feature, false);
    } catch (ParserConfigurationException e) {
        // This should catch a failed setFeature feature
        logger.info("ParserConfigurationException was thrown. The feature '" +
feature
+ "' is probably not supported by your XML processor.");
        ...
    }
}

try {
    // Add these as per Timothy Morgan's 2014 paper: "XML Schema, DTD, and
Entity Attacks"
    dbf.setXIncludeAware(false);
    dbf.setExpandEntityReferences(false);

    // As stated in the documentation, "Feature for Secure Processing (FSP)"
is the central mechanism that will
    // help you safeguard XML processing. It instructs XML processors, such as
parsers, validators,
    // and transformers, to try and process XML securely, and the FSP can be
used as an alternative to
    // dbf.setExpandEntityReferences(false); to allow some safe level of
Entity Expansion
    // Exists from JDK6.
    dbf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);

    // And, per Timothy Morgan: "If for some reason support for inline
DOCTYPEs are a requirement, then
    // ensure the entity settings are disabled (as shown above) and beware
that SSRF attacks
    // (http://cwe.mitre.org/data/definitions/918.html) and denial
    // of service attacks (such as billion laughs or decompression bombs via
"jar:") are a risk."

    // remaining parser logic
    ...
} catch (ParserConfigurationException e) {
    // This should catch a failed setFeature feature
    logger.info("ParserConfigurationException was thrown. The feature
'XMLConstants.FEATURE_SECURE_PROCESSING' "
+ " is probably not supported by your XML processor.");
    ...
} catch (SAXException e) {
    // On Apache, this should be thrown when disallowing DOCTYPE
    logger.warning("A DOCTYPE was passed into the XML document");
    ...
} catch (IOException e) {
    // XXE that points to a file that doesn't exist
    logger.error("IOException occurred, XXE may still possible: " +
e.getMessage());
}
```



```

    ...
}

// Load XML file or stream using a XXE agnostic configured parser...
DocumentBuilder safebuilder = dbf.newDocumentBuilder();

```

Xerces 1 Features:

- Do not include external entities by setting [this feature](#) to `false`.
- Do not include parameter entities by setting [this feature](#) to `false`.
- Do not include external DTDs by setting [this feature](#) to `false`.

Xerces 2 Features:

- Disallow an inline DTD by setting [this feature](#) to `true`.
- Do not include external entities by setting [this feature](#) to `false`.
- Do not include parameter entities by setting [this feature](#) to `false`.
- Do not include external DTDs by setting [this feature](#) to `false`.

Note: The above defenses require Java 7 update 67, Java 8 update 20, or above, because the countermeasures for `DocumentBuilderFactory` and `SAXParserFactory` are broken in earlier Java versions, per: [CVE-2014-6517](#).

XMLInputFactory (a StAX parser)

[StAX](#) parsers such as `XMLInputFactory` allow various properties and features to be set.

To protect a Java `XMLInputFactory` from XXE, disable DTDs (doctypes) altogether:

```

// This disables DTDs entirely for that factory
xmlInputFactory.setProperty(XMLInputFactory.SUPPORT_DTD, false);

```

or if you can't completely disable DTDs:

```

// This causes XMLStreamException to be thrown if external DTDs are accessed.
xmlInputFactory.setProperty(XMLConstants.ACCESS_EXTERNAL_DTD, "");
// disable external entities
xmlInputFactory.setProperty("javax.xml.stream.isSupportingExternalEntities",
false);

```

The setting `xmlInputFactory.setProperty(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");` is not required, as `XMLInputFactory` is dependent on `Validator` to perform XML validation against Schemas. Check the [Validator](#) section for the specific configuration.

Oracle DOM Parser

Follow [Oracle recommendation](#) e.g.:

```
// Extend oracle.xml.parser.v2.XMLParser
DOMParser domParser = new DOMParser();

// Do not expand entity references
domParser.setAttribute(DOMParser.EXPAND_ENTITYREF, false);

// dtdObj is an instance of oracle.xml.parser.v2.DTD
domParser.setAttribute(DOMParser.DTD_OBJECT, dtdObj);

// Do not allow more than 11 levels of entity expansion
domParser.setAttribute(DOMParser.ENTITY_EXPANSION_DEPTH, 12);
```

TransformerFactory

To protect a `javax.xml.transform.TransformerFactory` from XXE, do this:

```
TransformerFactory tf = TransformerFactory.newInstance();
tf.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
tf.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, "");
```

Validator

To protect a `javax.xml.validation.Validator` from XXE, do this:

```
SchemaFactory factory =
SchemaFactory.newInstance("http://www.w3.org/2001/XMLSchema");
factory.setProperty(XMLConstants.ACCESS_EXTERNAL_DTD, "");
factory.setProperty(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
Schema schema = factory.newSchema();
Validator validator = schema.newValidator();
validator.setProperty(XMLConstants.ACCESS_EXTERNAL_DTD, "");
validator.setProperty(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
```

SchemaFactory

To protect a `javax.xml.validation.SchemaFactory` from XXE, do this:

```
SchemaFactory factory =
SchemaFactory.newInstance("http://www.w3.org/2001/XMLSchema");
factory.setProperty(XMLConstants.ACCESS_EXTERNAL_DTD, "");
factory.setProperty(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
Schema schema = factory.newSchema(Source);
```

SAXTransformerFactory

To protect a `javax.xml.transform.sax.SAXTransformerFactory` from XXE, do this:

```
SAXTransformerFactory sf = SAXTransformerFactory.newInstance();
sf.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
sf.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, "");
sf.newXMLFilter(Source);
```

Note: Use of the following `XMLConstants` requires JAXP 1.5, which was added to Java in 7u40 and Java 8:

- `javax.xml.XMLConstants.ACCESS_EXTERNAL_DTD`
- `javax.xml.XMLConstants.ACCESS_EXTERNAL_SCHEMA`
- `javax.xml.XMLConstants.ACCESS_EXTERNAL_STYLESHEET`

XMLReader

To protect the Java `org.xml.sax.XMLReader` from an XXE attack, do this:

```
XMLReader reader = XMLReaderFactory.createXMLReader();
reader.setFeature("http://apache.org/xml/features/disallow-doctype-decl",
true);
// This may not be strictly required as DTDs shouldn't be allowed at all, per
previous line.
reader.setFeature("http://apache.org/xml/features/nonvalidating/load-external-
dtd", false);
reader.setFeature("http://xml.org/sax/features/external-general-entities",
false);
reader.setFeature("http://xml.org/sax/features/external-parameter-entities",
false);
```

SAXReader

To protect a Java `org.dom4j.io.SAXReader` from an XXE attack, do this:

```
saxReader.setFeature("http://apache.org/xml/features/disallow-doctype-decl",
true);
saxReader.setFeature("http://xml.org/sax/features/external-general-entities",
false);
saxReader.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
```

If your code does not have all of these lines, you could be vulnerable to an XXE attack.

SAXBuilder

To protect a Java `org.jdom2.input.SAXBuilder` from an XXE attack, disallow DTDs (doctypes) entirely:

```
SAXBuilder builder = new SAXBuilder();
builder.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
Document doc = builder.build(new File(fileName));
```

Alternatively, if DTDs can't be completely disabled, disable external entities and entity expansion:

```
SAXBuilder builder = new SAXBuilder();
builder.setFeature("http://xml.org/sax/features/external-general-entities", false);
builder.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
builder.setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd", false);
builder.setExpandEntities(false);
Document doc = builder.build(new File(fileName));
```

No-op EntityResolver

For APIs that take an `EntityResolver`, you can neutralize an XML parser's ability to resolve entities by [supplying a no-op implementation](#):

```
public final class NoOpEntityResolver implements EntityResolver {
    public InputSource resolveEntity(String publicId, String systemId) {
        return new InputSource(new StringReader(""));
    }
}

// ...

xmlReader.setEntityResolver(new NoOpEntityResolver());
documentBuilder.setEntityResolver(new NoOpEntityResolver());
```

or more simply:

```
EntityResolver noop = (publicId, systemId) -> new InputSource(new
StringReader(""));
xmlReader.setEntityResolver(noop);
documentBuilder.setEntityResolver(noop);
```

JAXB Unmarshaller

You should ensure that the source to the `unmarshal` function of

`javax.xml.bind.Unmarshaller` is `javax.xml.stream.XMLStreamReader` that was generated using `javax.xml.stream.XMLInputFactory` with safe properties, i.e.

`XMLInputFactory.SUPPORT_DTD` and `XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES` set to `false`. For example:

```
File file = new File(xmlPath);
XMLInputFactory xif = XMLInputFactory.newFactory();
xif.setProperty(XMLInputFactory.SUPPORT_DTD, false);
xif.setProperty(XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false);
XMLStreamReader xsr = null;
try {
    xsr = xif.createXMLStreamReader(new StreamSource(file));
} catch (XMLStreamException e) {
    throw new RuntimeException(e);
}
Unmarshaller um = jc.createUnmarshaller();
um.unmarshal(xsr);
```

Note that both the `createXMLStreamReader` and `unmarshal` methods have several overloads with various source types, so you need to pick the right one and do a possible conversion.

XPathExpression

Since `javax.xml.xpath.XPathExpression` can not be configured securely by itself, the untrusted data must be parsed through another securable XML parser first.

For example:

```
DocumentBuilderFactory df = DocumentBuilderFactory.newInstance();
df.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
df.setAttribute(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
DocumentBuilder builder = df.newDocumentBuilder();
String result = new XPathExpression().evaluate( builder.parse(
    new ByteArrayInputStream(xml.getBytes())) );
```

java.beans.XMLDecoder

The `readObject()` method in this class is fundamentally unsafe.

Not only is the XML it parses subject to XXE, but the method can be used to construct any Java object, and [execute arbitrary code as described here](#).

And there is no way to make use of this class safe except to trust or properly validate the input being passed into it.

As such, we'd strongly recommend completely avoiding the use of this class and replacing it with a safe or properly configured XML parser as described elsewhere in this cheat sheet.

Other XML Parsers

There are many third-party libraries that parse XML either directly or through their use of other libraries. Please test and verify their XML parser is secure against XXE by default. If the parser is not secure by default, look for flags supported by the parser to disable all possible

external resource inclusions like the examples given above. If there's no control exposed to the outside, make sure the untrusted content is passed through a secure parser first and then passed to insecure third-party parser similar to how the Unmarshaller is secured.

Spring Framework MVC/OXM XXE Vulnerabilities

**Some XXE vulnerabilities were found in [Spring OXM](#) and [Spring MVC](#) . The following versions of the Spring Framework are vulnerable to XXE:

- **3.0.0 to 3.2.3** (Spring OXM & Spring MVC)
- **4.0.0.M1** (Spring OXM)
- **4.0.0.M1-4.0.0.M2** (Spring MVC)

There were other issues as well that were fixed later, so to fully address these issues, Spring recommends you upgrade to Spring Framework 3.2.8+ or 4.0.2+.

For Spring OXM, this is referring to the use of `org.springframework.xml.jaxb.Jaxb2Marshaller`.

Note that the CVE for Spring OXM specifically indicates that two XML parsing situations are up to the developer to get right, and the other two are the responsibility of Spring and were fixed to address this CVE.

Here's what they say:

Two situations developers must handle:

- For a `DOMSource` , the XML has already been parsed by user code and that code is responsible for protecting against XXE.
- For a `StAXSource` , the `XMLStreamReader` has already been created by user code and that code is responsible for protecting against XXE.

The issue Spring fixed:

For `SAXSource` and `StreamSource` instances, Spring processed external entities by default thereby creating this vulnerability.

Here's an example of using a `StreamSource` that was vulnerable, but is now safe, if you are using a fixed version of Spring OXM or Spring MVC:

```
import org.springframework.xml.jaxb.Jaxb2Marshaller;
import org.springframework.xml.jaxb.Jaxb2Marshaller;

Jaxb2Marshaller marshaller = new Jaxb2Marshaller();
// Must cast return Object to whatever type you are unmarshalling
marshaller.unmarshal(new StreamSource(new
StringReader(some_string_containing_XML));
```

So, per the [Spring OXM CVE writeup](#), the above is now safe. But if you were to use a DOMSource or StAXSource instead, it would be up to you to configure those sources to be safe from XXE.

Castor

Castor is a data binding framework for Java. It allows conversion between Java objects, XML, and relational tables. The XML features in Castor prior to version 1.3.3 are vulnerable to XXE, and should be upgraded to the latest version. For additional information, check the official [XML configuration file](#)

.NET

Up-to-date information for XXE injection in .NET is taken directly from the [web application of unit tests by Dean Fleming](#), which covers all currently supported .NET XML parsers, and has test cases that demonstrate when they are safe from XXE injection and when they are not, but these tests are only with injection from file and not direct DTD (used by DoS attacks).

For DoS attacks using a direct DTD (such as the [Billion laughs attack](#)), a [separate testing application from Josh Grossman at Bounce Security](#) has been created to verify that .NET >=4.5.2 is safe from these attacks.

Previously, this information was based on some older articles which may not be 100% accurate including:

- [James Jardine's excellent .NET XXE article.](#)
- [Guidance from Microsoft on how to prevent XXE and XML Denial of Service in .NET.](#)

Overview of .NET Parser Safety Levels

****Below is an overview of all supported .NET XML parsers and their default safety levels. More details about each parser are included after this list.**

****XDocument (LINQ to XML)**

This parser is protected from external entities at .NET Framework version 4.5.2 and protected from Billion Laughs at version 4.5.2 or greater, but it is uncertain if this parser is protected from Billion Laughs before version 4.5.2.

XmlDocument, XmlTextReader, XPathNavigator default safety levels

These parsers are vulnerable to external entity attacks and Billion Laughs at versions below version 4.5.2 but protected at versions equal or greater than 4.5.2.

XmlDictionaryReader, XmlNodeReader, XmlReader default safety levels

These parsers are not vulnerable to external entity attacks or Billion Laughs before or after version 4.5.2. Also, at or greater than versions $\geq 4.5.2$, these libraries won't even process the in-line DTD by default. Even if you change the default to allow processing a DTD, if a DoS attempt is performed an exception will still be thrown as documented above.

ASP.NET

ASP.NET applications $\geq$.NET 4.5.2 must also ensure setting the `<httpRuntime targetFramework="..." />` in their `Web.config` to $\geq 4.5.2$ or risk being vulnerable regardless of the actual .NET version. Omitting this tag will also result in unsafe-by-default behavior.

For the purpose of understanding the above table, the `.NET Framework Version` for an ASP.NET applications is either the .NET version the application was build with or the `httpRuntime's targetFramework` (`Web.config`), **whichever is lower**.

This configuration tag should not be confused with a similar configuration tag: `<compilation targetFramework="..." />` or the assemblies / projects `targetFramework`, which are **not** sufficient for achieving secure-by-default behaviour as advertised in the above table.

LINQ to XML

Both the `XElement` and `XDocument` objects in the `System.Xml.Linq` library are safe from XXE injection from external file and DoS attack by default. `XElement` parses only the elements within the XML file, so DTDs are ignored altogether. `XDocument` has `XmlResolver` **disabled by default** so it's safe from SSRF. Whilst DTDs are **enabled by default**, from Framework versions $\geq 4.5.2$, it is **not** vulnerable to DoS as noted but it may be vulnerable in earlier Framework versions. For more information, see [Microsoft's guidance on how to prevent XXE and XML Denial of Service in .NET](#)

XmlDictionaryReader

`System.Xml.XmlDictionaryReader` is safe by default, as when it attempts to parse the DTD, the compiler throws an exception saying that "CDATA elements not valid at top level of an XML document". It becomes unsafe if constructed with a different unsafe XML parser.

XmlDocument

Prior to .NET Framework version 4.5.2, `System.Xml.XmlDocument` is unsafe by default. The `XmlDocument` object has an `XmlResolver` object within it that needs to be set to null in versions prior to 4.5.2. In versions 4.5.2 and up, this `XmlResolver` is set to null by default.

The following example shows how it is made safe:


```
static void LoadXML()
{
    string xxePayload = "<!DOCTYPE doc [<!ENTITY win SYSTEM
'file:///C:/Users/testdata2.txt'>]>"
        + "<doc>&win;</doc>";
    string xml = "<?xml version='1.0' ?>" + xxePayload;

    XmlDocument xmlDoc = new XmlDocument();
    // Setting this to NULL disables DTDs - Its NOT null by default.
    xmlDoc.XmlResolver = null;
    xmlDoc.LoadXml(xml);
    Console.WriteLine(xmlDoc.InnerText);
    Console.ReadLine();
}
```

For .NET Framework version $\geq 4.5.2$, this is safe by default.

`XmlDocument` can become unsafe if you create your own nonnull `XmlResolver` with default or unsafe settings. If you need to enable DTD processing, instructions on how to do so safely are described in detail in the [referenced MSDN article](#).

XmlNodeReader

`System.Xml.XmlNodeReader` objects are safe by default and will ignore DTDs even when constructed with an unsafe parser or wrapped in another unsafe parser.

XmlReader

`System.Xml.XmlReader` objects are safe by default.

They are set by default to have their `ProhibitDtd` property set to false in .NET Framework versions 4.0 and earlier, or their `DtdProcessing` property set to Prohibit in .NET versions 4.0 and later.

Additionally, in .NET versions 4.5.2 and later, the `XmlReaderSettings` belonging to the `XmlReader` has its `XmlResolver` set to null by default, which provides an additional layer of safety.

Therefore, `XmlReader` objects will only become unsafe in version 4.5.2 and up if both the `DtdProcessing` property is set to Parse and the `XmlReaderSetting`'s `XmlResolver` is set to a nonnull `XmlResolver` with default or unsafe settings. If you need to enable DTD processing, instructions on how to do so safely are described in detail in the [referenced MSDN article](#).

XmlTextReader

`System.Xml.XmlTextReader` is **unsafe** by default in .NET Framework versions prior to 4.5.2. Here is how to make it safe in various .NET versions:

Prior to .NET 4.0

In .NET Framework versions prior to 4.0, DTD parsing behavior for `XmlReader` objects like `XmlTextReader` are controlled by the Boolean `ProhibitDtd` property found in the `System.Xml.XmlReaderSettings` and `System.Xml.XmlTextReader` classes.

Set these values to true to disable inline DTDs completely.

```
XmlTextReader reader = new XmlTextReader(stream);  
// NEEDED because the default is FALSE!!  
reader.ProhibitDtd = true;
```

.NET 4.0 - .NET 4.5.2

In .NET Framework version 4.0, DTD parsing behavior has been changed. The `ProhibitDtd` property has been deprecated in favor of the new `DtdProcessing` property.

However, they didn't change the default settings so `XmlTextReader` is still vulnerable to XXE by default.

Setting `DtdProcessing` to `Prohibit` causes the runtime to throw an exception if a `<!DOCTYPE>` element is present in the XML.

To set this value yourself, it looks like this:

```
XmlTextReader reader = new XmlTextReader(stream);  
// NEEDED because the default is Parse!!  
reader.DtdProcessing = DtdProcessing.Prohibit;
```

Alternatively, you can set the `DtdProcessing` property to `Ignore`, which will not throw an exception on encountering a `<!DOCTYPE>` element but will simply skip over it and not process it. Finally, you can set `DtdProcessing` to `Parse` if you do want to allow and process inline DTDs.

.NET 4.5.2 and later

In .NET Framework versions 4.5.2 and up, `XmlTextReader`'s internal `XmlResolver` is set to null by default, making the `XmlTextReader` ignore DTDs by default. The `XmlTextReader` can become unsafe if you create your own nonnull `XmlResolver` with default or unsafe settings.

XPathNavigator

`System.Xml.XPath.XPathNavigator` is **unsafe** by default in .NET Framework versions prior to 4.5.2.

This is due to the fact that it implements `IXPathNavigable` objects like `XmlDocument`, which are also unsafe by default in versions prior to 4.5.2.

You can make `XPathNavigator` safe by giving it a safe parser like `XmlReader` (which is safe by default) in the `XPathDocument`'s constructor.

Here is an example:

```
XmlReader reader = XmlReader.Create("example.xml");
XPathDocument doc = new XPathDocument(reader);
XPathNavigator nav = doc.CreateNavigator();
string xml = nav.InnerXml.ToString();
```

For .NET Framework version $\geq 4.5.2$, `XPathNavigator` is **safe by default**.

XslCompiledTransform

`System.Xml.Xsl.XslCompiledTransform` (an XML transformer) is safe by default as long as the parser it's given is safe.

It is safe by default because the default parser of the `Transform()` methods is an `XmlReader`, which is safe by default (per above).

[The source code for this method is here.](#)

Some of the `Transform()` methods accept an `XmlReader` or `IXPathNavigable` (e.g., `XmlDocument`) as an input, and if you pass in an unsafe XML Parser then the `Transform` will also be unsafe.

iOS

libxml2

iOS includes the C/C++ libxml2 library described above, so that guidance applies if you are using libxml2 directly.

However, the version of libxml2 provided up through iOS6 is prior to version 2.9 of libxml2 (which protects against XXE by default).

NSXMLDocument

iOS also provides an `NSXMLDocument` type, which is built on top of libxml2.

However, `NSXMLDocument` provides some additional protections against XXE that aren't available in libxml2 directly.

Per the 'NSXMLDocument External Entity Restriction API' section of this [page](#):

- iOS4 and earlier: All external entities are loaded by default.

- iOS5 and later: Only entities that don't require network access are loaded. (which is safer)

However, to completely disable XXE in an `NSXMLDocument` in any version of iOS you simply specify `NSXMLNodeLoadExternalEntitiesNever` when creating the `NSXMLDocument`.

PHP

When using the default XML parser (based on libxml2), PHP 8.0 and newer **prevent XXE by default**.

For PHP versions prior to 8.0, per [the PHP documentation](#), the following should be set when using the default PHP XML parser in order to prevent XXE:

```
libxml_set_external_entity_loader(null);
```

A description of how to abuse this in PHP is presented in a good [SensePost article](#) describing a cool PHP based XXE vulnerability that was fixed in Facebook.

Python

The Python 3 official documentation contains a section on [xml vulnerabilities](#). As of the 1st January 2020 Python 2 is no longer supported, however the Python website still contains [some legacy documentation](#).

The table below shows you which various XML parsing modules in Python 3 are vulnerable to certain XXE attacks.

Attack Type	sax	etree	minidom	pulldom	xmlrpc
Billion Laughs	Vulnerable	Vulnerable	Vulnerable	Vulnerable	Vulnerab
Quadratic Blowup	Vulnerable	Vulnerable	Vulnerable	Vulnerable	Vulnerab
External Entity Expansion	Safe	Safe	Safe	Safe	Safe
DTD Retrieval	Safe	Safe	Safe	Safe	Safe
Decompression Bomb	Safe	Safe	Safe	Safe	Vulnerab

To protect your application from the applicable attacks, [two packages](#) exist to help you sanitize your input and protect your application against DDoS and remote attacks.

Semgrep Rules

[Semgrep](#) is a command-line tool for offline static analysis. Use pre-built or custom rules to enforce code and security standards in your codebase.

Java

Below are the rules for different XML parsers in Java

Digester

Identifying XXE vulnerability in the `org.apache.commons.digester3.Digester` library Rule can be played here <https://semgrep.dev/s/salecharohit:xxe-Digester>

DocumentBuilderFactory

Identifying XXE vulnerability in the `javax.xml.parsers.DocumentBuilderFactory` library Rule can be played here <https://semgrep.dev/s/salecharohit:xxe-dbf>

SAXBuilder

Identifying XXE vulnerability in the `org.jdom2.input.SAXBuilder` library Rule can be played here <https://semgrep.dev/s/salecharohit:xxe-saxbuilder>

SAXParserFactory

Identifying XXE vulnerability in the `javax.xml.parsers.SAXParserFactory` library Rule can be played here <https://semgrep.dev/s/salecharohit:xxe-SAXParserFactory>

SAXReader

Identifying XXE vulnerability in the `org.dom4j.io.SAXReader` library Rule can be played here <https://semgrep.dev/s/salecharohit:xxe-SAXReader>

XMLInputFactory

Identifying XXE vulnerability in the `javax.xml.stream.XMLInputFactory` library Rule can be played here <https://semgrep.dev/s/salecharohit:xxe-XMLInputFactory>

XMLReader

Identifying XXE vulnerability in the `org.xml.sax.XMLReader` library Rule can be played here <https://semgrep.dev/s/salecharohit:xxe-XMLReader>

References

- [XXE by InfoSecInstitute](#)
- [OWASP Top 10-2017 A4: XML External Entities \(XXE\)](#)
- [Timothy Morgan's 2014 paper: "XML Schema, DTD, and Entity Attacks"](#)
- [FindSecBugs XXE Detection](#)
- [XXEbugFind Tool](#)
- [Testing for XML Injection](#)